MPLab Research Project: Multi-Time Step SDP Based Energy Management Script - Implementation In C And Embedded Platforms

Executed By
Anurag S Chatterjee

Project Supervisor
Ariana Haghighi

Project Sponsor
Navid Rahbariasr

Project Dates
Jan 30, 2026 - May 15, 2026

Presentation Overview

01 **Introduction/Background**

Context/Motivation, Problem Statement, Objectives, Hypothesis And Research Approach

02 **Methodology**

Implementation, System Design, Setup, Technical Execution, Challenges Faced And Important Learnings

03 **Analysis**

Results Obtained, Benchmarking, Performance Analysis, Findings And Recommendations

SECTION 01

Introduction

Context, Problem Statement, Objectives, Hypothesis & Research Approach

Problem Statement

Implementation of convexified power flow optimization on a Raspberry Pi.

Objectives

Development of a high-performance C-based SDP solver for Raspberry Pi deployment.

Hypothesis

SDP relaxation provides optimal energy management with minimal computational overhead.

Mathematical Foundation

Convex reformulation via lifting and rank relaxation for semidefinite programming.

Research Approach

Iterative prototyping from MATLAB simulation to ARM-based C execution.

Project Purpose, Goals And Strategic Objectives

Project Purpose

Translate SDP energy optimization from MATLAB/CVX to C for Raspberry Pi deployment. Assess Simulink reintegration for processor in the loop simulations, faster execution, and resource-constrained system deployment.

Goals

- Debug multi-time step SDP code into C
- Implement Ethernet-based optimization
- Investigate QP/CVXGEN feasibility on Raspberry Pi
- Define future implementation paths

Strategic Objectives

- Study existing C-SDP & CVX scripts
- Compile & deploy C, CVXPY and CVXPYGEN on Raspberry Pi
- Benchmark runtime, solver accuracy & memory usage
- Assess numerical stability on platforms
- Document feasibility & technical outcomes

Background: The Energy Management Problem

Problem Significance

⚡ **Solar Intermittency:** Power generation (W) varies hour to hour across the 5-bus microgrid.

🔋 **Battery Balancing:** Must optimize charging/discharging efficiencies ($\eta c, \eta d \in (0,1)$).

⚠️ **Non-Convex Constraints:** Battery cannot charge & discharge simultaneously: $p^c(t) \cdot p^d(t) = 0$.

📐 **DC Power Flow:** Voltage bilinearities (Ohm's law) make the full OPF problem non-convex.

System Parameters

- T = 24 hourly timesteps, Δt = 1 hr
- Batteries: 500 kWh each (Buses 3, 4 & 5)

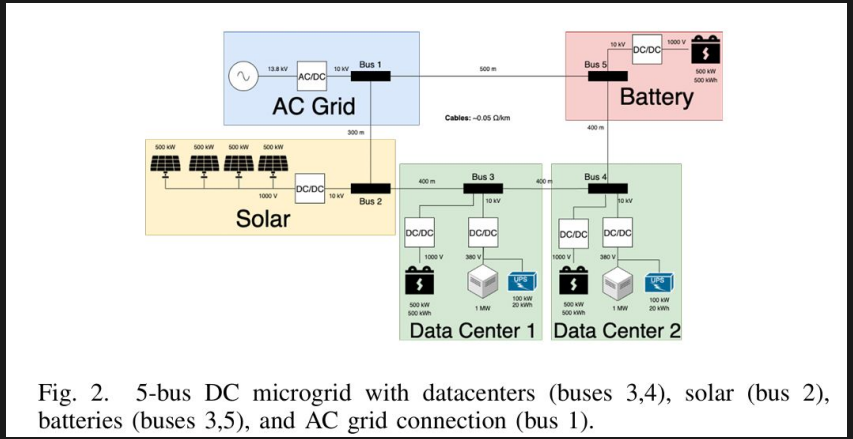


Fig. 2. 5-bus DC microgrid with datacenters (buses 3,4), solar (bus 2), batteries (buses 3,5), and AC grid connection (bus 1).

Figure above adapted from Haghighi et al.: battery complementarity constraints paper

Motivation: Why This Research?

The Embedded Deployment Gap

State-of-the-art SDP solvers (MATLAB/CVX) run on powerful workstations.

Deploying them on Raspberry Pi and embedded microcontrollers for real-time microgrid control is a critical engineering challenge.

Speed is Critical

Pure Python CVXPY takes ~1-2 seconds on Raspberry Pi. Real-time control needs sub-100 ms decisions.

25× Speedup

CVXPYGEN achieves ~55 ms (0.055s) -enabling embedded deployment for the first time.

Novel Battery Complementarity

Implements result from Haghighi et al.: battery complementarity constraints $p^c(t) \cdot p^d(t) = 0$ are implicitly satisfied when penalty costs are added.

✓ Removes need for expensive mixed integer solvers

Problem Statement And Key Questions To Ask

How can the SDP-based multi-period DC microgrid optimal power flow problem be translated from MATLAB/CVX into C and deployed on a Raspberry Pi 5 while preserving optimality and achieving real-time performance?

1. C Reproduction of MATLAB/CVX Optimality?

Raw C code must manually define cost matrix C, 19 equality constraints, and 27×27 PSD variable. Does the optimal objective match MATLAB's CVX abstraction?

2. Is CVXGEN Applicable to SDP Problems?

CVXGEN generates C for QPs/LPs only. It cannot handle matrix variables $V[t] \geq 0$ or PSD constraints.

3. Can CVXPYGEN Bridge Python → C for SDP?

CVXPYGEN converts CVXPY scripts into standalone C solvers. Evaluating this path on Raspberry Pi 5 is a core research question.

4. Real-World Embedded System Limits?

Memory, execution time and latency impose hard constraints. Analysis across 304 state of charge, charging and discharging values to ensure system robustness.

Workflow/Methodology

Phase 1: Literature Review

Weeks 1–2

- Read Haghighi et al. paper
- Review CSDP documentation
- Analyze MATLAB/CVX codebase structure
- Local runs: simple_sdp and dcpf_sdp

Phase 2: Feasibility Analysis

Week 3

- CVXGEN evaluation (Incompatible)
- CVXPY on Raspberry Pi
- Debug simple_sdp.c constraints

Phase 3: C Code Generation

Weeks 4–5

- Generate C code via CVXPYGEN
- DCPF_multi_convex.py mapping
- Array mapping: SOC, demand, solar

Phase 4: Embedded Deployment

Weeks 5–7

- Raspberry Pi setup (WiFi to Ethernet TCP/IP)
- Real-time send/receive of optimization profiles
- Integration of results processing and plot results (SOC, Demand, Power Profile, Runtime)

Phase 5: Benchmarking & Analysis

Weeks 7–9

- 304 fixed permutations (SOC 5–95%, eta_c and eta_d 80–95%)
- Compute time data acquisition and solver runtime
- Cost objective difference analysis & plotting

Project Execution Milestones

Milestone Task & Summary	Target Date	Status
Phase 1 Results Reading, SDP Code Review & Documentation	Feb 15	✓ Done
CVXGEN Feasibility Study & Raspberry Pi Setup	Feb 22	✓ Done
CVXPYGEN C-Code Generation & Multi-step Debugging	Mar 15	✓ Done
C-Source Mapping (SOC, Solar, Demand Profiles)	Mar 24	✓ Done
Ethernet Real-Time Data Interface Implementation And Benchmarking	Mar 31	✓ Done
Convex Optimization Result Unwrapping & MATLAB Integration	Apr 14	✓ Done
Fixed Permutations Scenario Benchmarking (~304 runs)	Apr 21	✓ Done
Non-Convex Vs Convex Cost Debugging, Critical Analysis, Runtime Graphs & Scripting	May 1	✓ Done
Simulink Integration & Final Documentation Submission	May 15	Remaining

SECTION 02

Methodology

Implementation, System Design, Setup, Technical Execution, Challenges & Learnings

WK 1–2

Literature Review

In-depth reading and initial local environment setup.

WK 3

RPi Feasibility

CVXGEN study, CVXPY deployment and testing on Raspberry Pi 5.

WK 4

C-Solver

CVXPYGEN code generation and benchmarking.

WK 5

Data Mapping

Battery SOC and solar profile integration.

WK 6

Ethernet Communication

Real-time interface and system validation.

SECTION 03

Analysis

Results Obtained, Benchmarking, Performance Analysis, Findings And Recommendations

Results Obtained

Validation of C-based solver accuracy against MATLAB benchmarks for multi-step scenarios.

Benchmarking

Execution time comparison between Raspberry Pi 4 and standard x86 workstation environments.

Performance Analysis

Evaluation of memory footprint and CPU utilization during peak optimization cycles.

Key Findings

Verification of SDP relaxation tightness and successful avoidance of non-convex local minima.

Recommendations

Strategic paths for future C/S-Function integration within hardware-in-the-loop (HIL) frameworks.

Results Obtained: Experiment 1: Fixed-Parameter Optimisation And Plotting Pipeline

Automated data flow and solver benchmarking over real-time Ethernet interface

Fixed Parameters: Einit = 0.01 p.u. · ηc = 0.99 · ηd = 0.99 · T = 24 hourly timesteps · 5-bus DC microgrid

💻 **MATLAB Client**

Laptop (Client)

- **Loads solar_power_data.mat**: 5 buses × 24 hours real measured profile
- **Scales** to p.u. (Sbase = 5 MW)
- **Builds load** (Buses 3 & 4: 1 MW constant)
- **Sets battery SOC**: Einit = 10%
- **Packages & Sends** JSON payload via Ethernet (TCP/IP, Port 5000)
- **Saves results** for MATLAB plotting

🍓 **Raspberry Pi**

Server (Computation)

- **Sequential Solving**:
  1. **CVXPY** (Python/Clarabel)
  2. **CVXPYGEN** (Pre-compiled C)
- **Extracts arrays**: power p, charging pc, discharging pd, SOC E
- **Returns JSON** results to laptop

📊 **MATLAB Plotter**

plot_sdp_results.m

- **Generates 7 Figures** automatically:
  - Power Generation Per Bus (Bar)
  - SOC Trajectory (Line)
  - Dispatch Overview (Subplots)
  - Solver Runtime Benchmarks

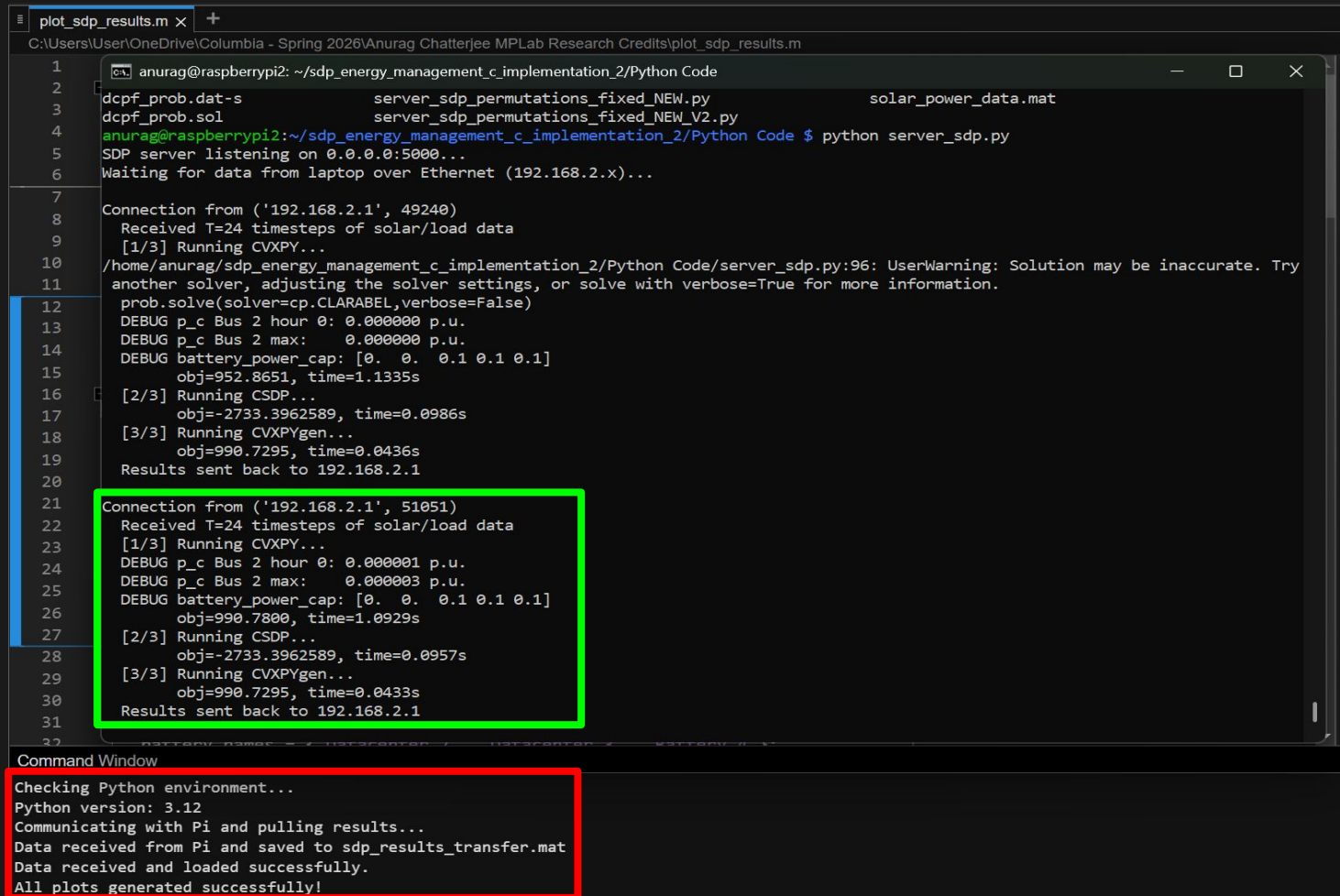
✓ **Key Outcome**: Full validated pipeline from single MATLAB run

Experimental Setup - Both Experiments 1 And 2

Experiment 1: Demo

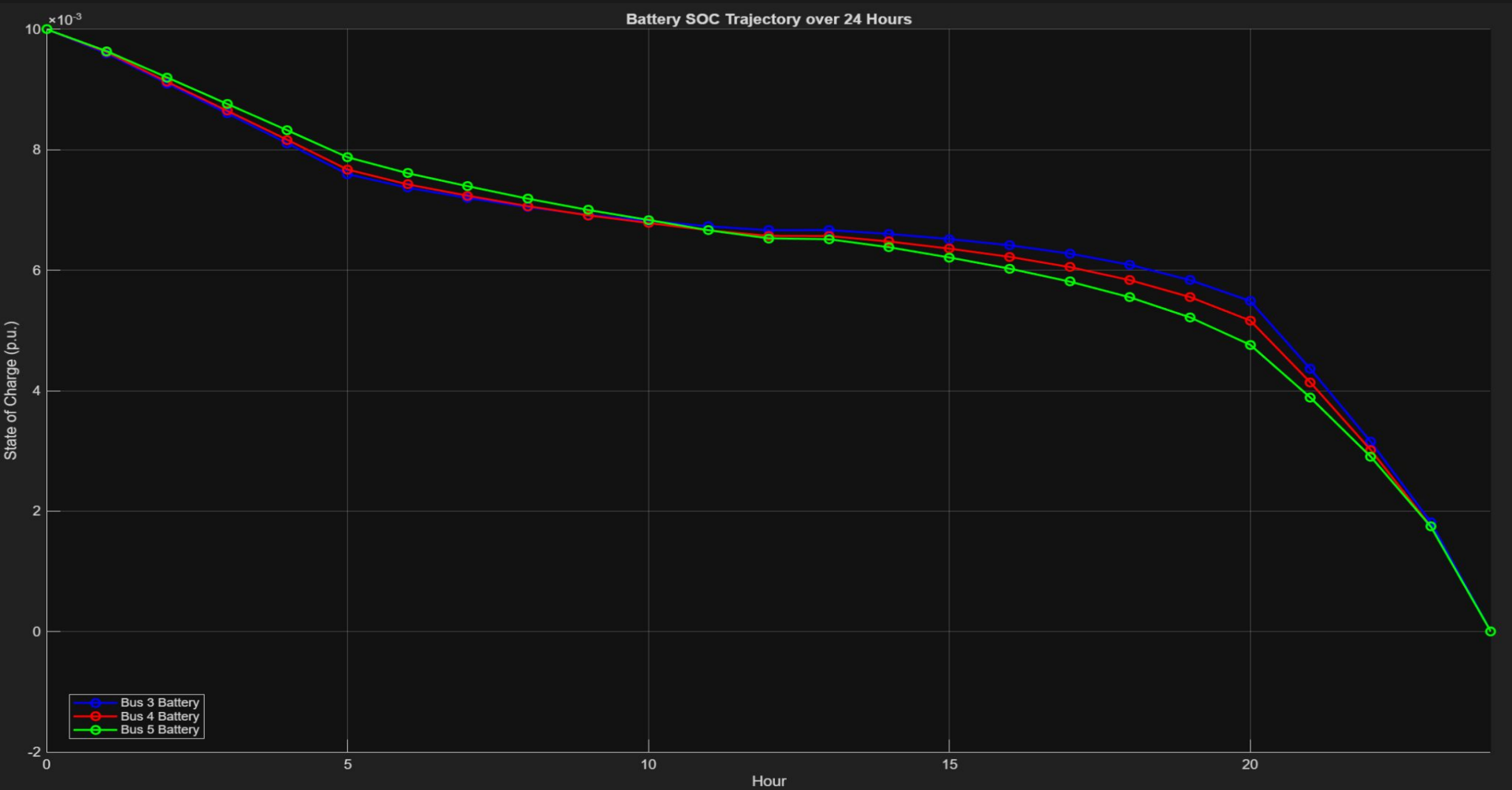
Server (Raspberry Pi)

MATLAB Interface

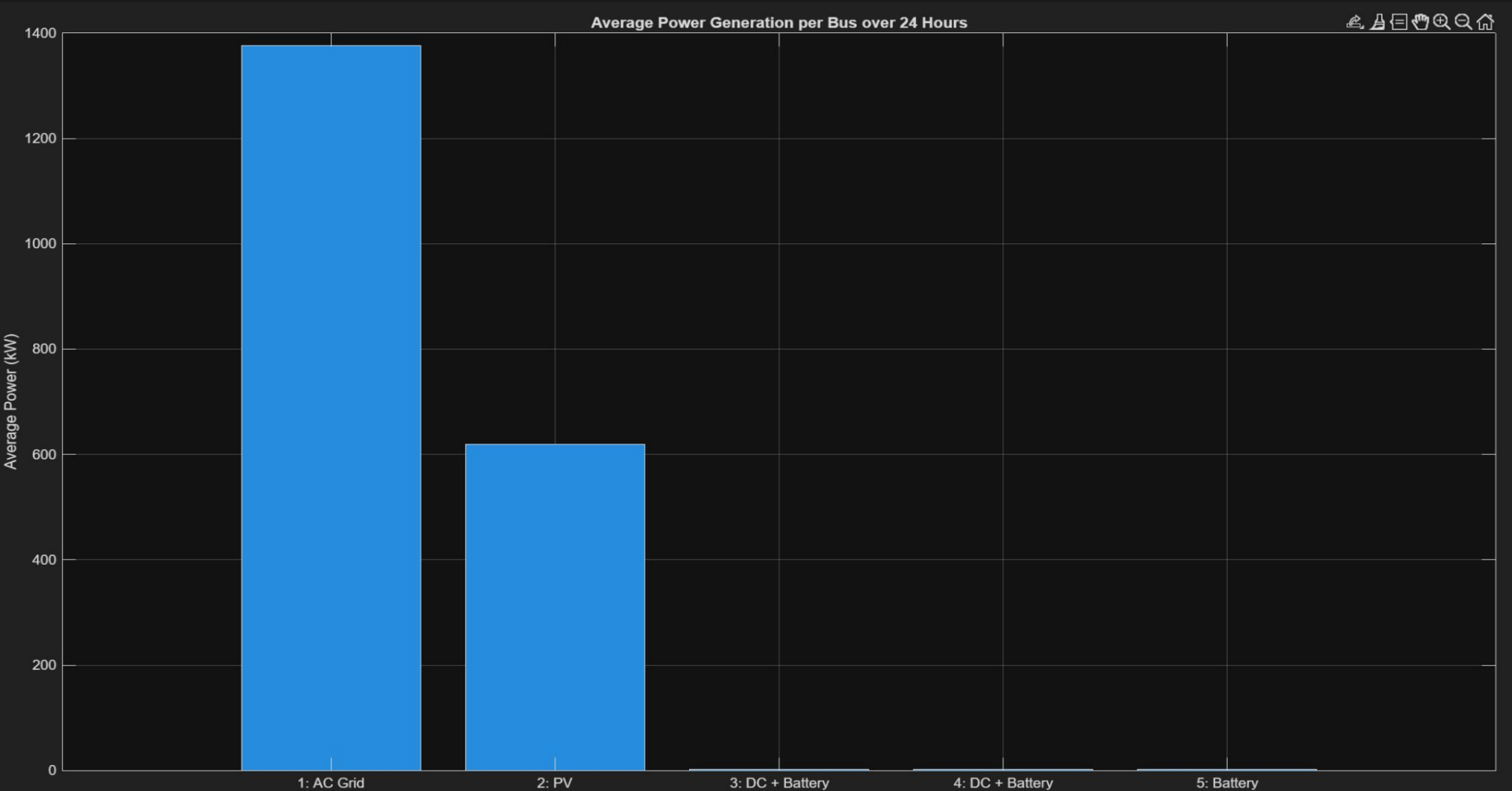


All Battery Buses - Dispatch Overview

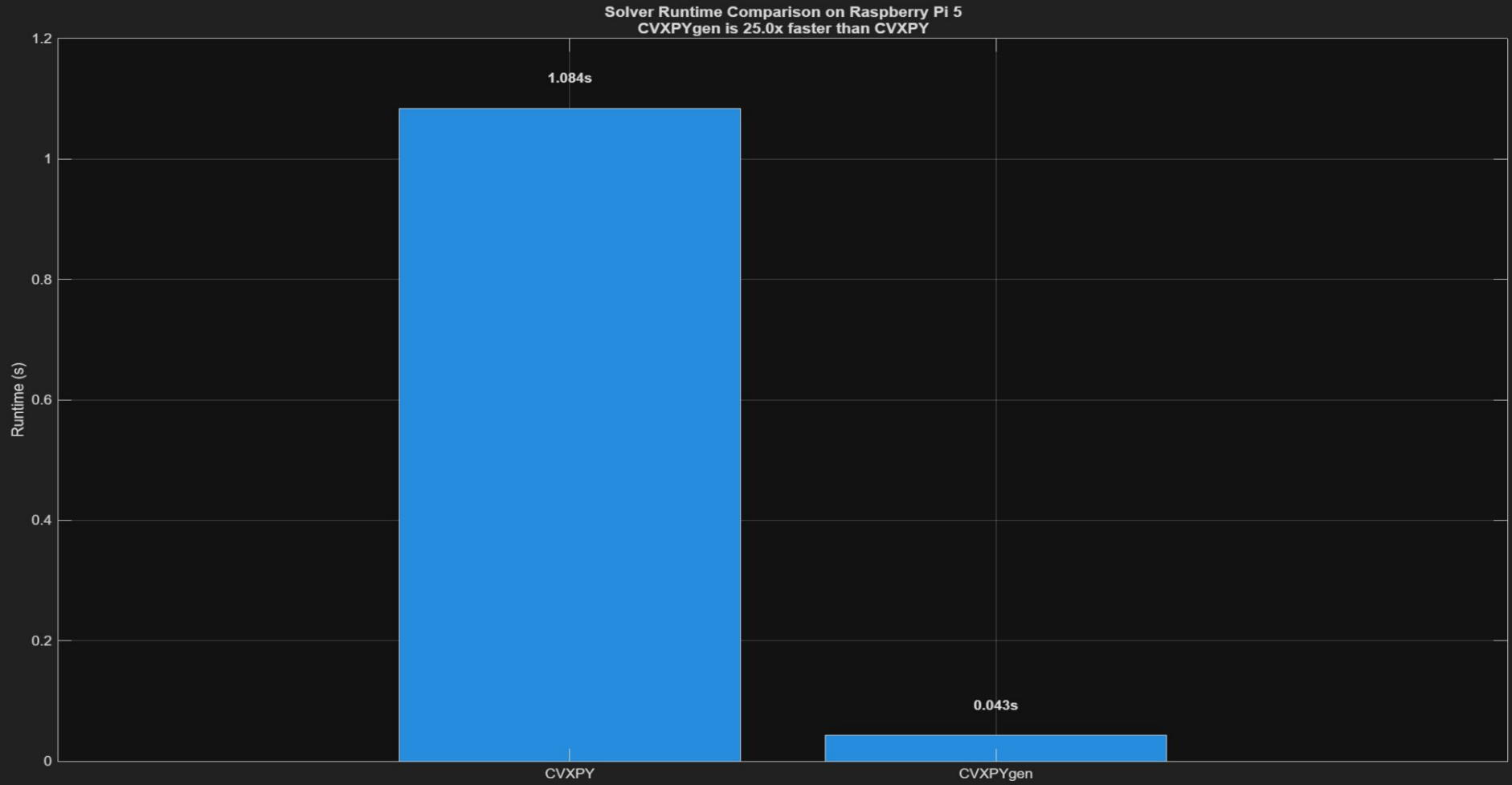
Battery SOC Trajectory



Power Generation Per Bus



Solver Runtime Benchmarking On Raspberry Pi



Results: Experiment 2: 304 Fixed Permutations Test And Performance Analysis

Parameter Space Exploration: η_c (4) × η_d (4) × Einit (19) = 304 total permutations

η_c: 0.80 → 0.95 (step 0.05) | η_d: 0.80 → 0.95 (step 0.05) | Einit: 0.05 (5% full at 0.005 p.u) → 0.95 (95% full at 0.095 p.u with step 0.05)

💻 MATLAB Client

Drives Permutations

- **Iterates** all η_c, η_d, Einit combinations
- **pyrunfile()** triggers Python client
- **Waits** for sdp_results_transfer.mat
- **Filters** failed runs silently
- **Accumulates** 304 result arrays
- **Statistical Plotting** after run

🍓 Raspberry Pi

Per-Permutation Solve

- **Solvers**: CVXPY and CVXPYGEN
- **Scaling**: Einit updated via cp.Parameter() each run
- **[VERIFY]** check: objective vs Einit
- **[DIFF]** check: |CVXPY − CVXPYGEN|
- **Sends JSON** results per loop

📊 MATLAB Plotter

Statistical/Performance Analysis

- **Panel 1**: Cost diff (%) histogram
- **Panel 2**: Runtime box plot (Pi)
- **Panel 3**: Objective diff distribution which is **approximately 0**
- **Panel 4**: p_c × p_d complementarity which is **negligible, confirming theorem**

✓ **Conclusion:**: **Zero relaxation/objective cost gap** across all scenarios (**CVXPY objective cost > CVXPYGEN objective cost**)

💡 **Einit Effect:** Higher Einit → more energy → lower generation cost. Verified for all 304 runs.

Experiment 2: Demo

Server (Raspberry Pi)

MATLAB Interface

DCPF_permutation_script_simplified_fixed_from_supervisor_code.m

C:\Users\User\OneDrive\Columbia - Spring 2026\Anurag Chatterjee MPLab Research Credits\DCPF_permutation_script_simplified_fixed_from_supervisor_code.m

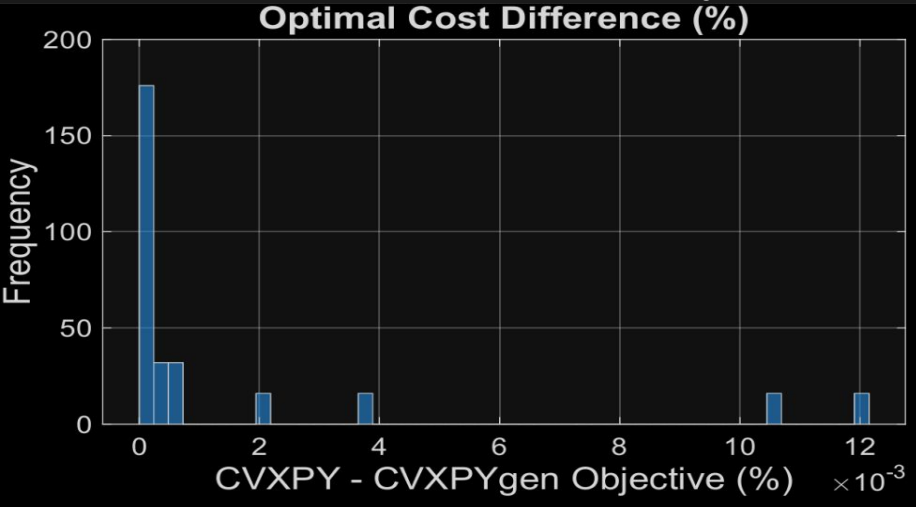
anurag@raspberrypi2: ~/sdp_energy_management_c_implementation_2/Python Code

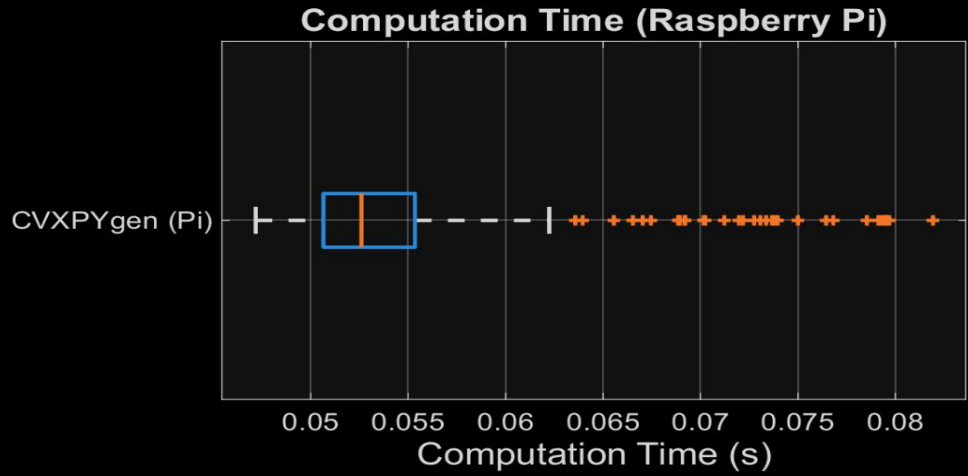
```
  equilibrate: on, min_scale = 1.0e-4, max_scale = 1.0e4
               max iter = 10

iter    pcost        dcost       gap       pres      dres      k/t        μ       step
-------------------------------------------------------------------------------------------
  0  +1.7389e+05  -2.6058e+05  2.50e+00  2.87e-01  1.34e-01  1.00e+00  1.20e+04   ------
  1  +1.2993e+05  +8.6881e+03  1.40e+01  6.65e-02  2.42e-02  1.37e+03  2.33e+03  8.84e-01
  2  -3.6138e+03  -2.8424e+04  6.87e+00  1.32e-02  4.70e-03  9.45e+02  6.70e+02  8.03e-01
  3  -6.5781e+03  -1.1292e+04  7.17e-01  2.46e-03  8.96e-04  1.36e+02  1.59e+02  8.63e-01
  4  -4.0392e+02  -7.2983e+02  8.07e-01  1.70e-04  6.17e-05  5.24e+00  1.18e+01  9.54e-01
  5  -8.2147e+01  -1.4871e+02  8.10e-01  3.47e-05  1.29e-05  1.03e+00  3.43e+00  7.26e-01
  6  -1.1211e+02  -2.2653e+02  1.02e+00  6.02e-05  2.18e-05  2.89e+00  2.50e+00  5.18e-01
  7  +4.9376e+01  -4.3920e+01  2.12e+00  4.89e-05  1.78e-05  2.03e+00  2.37e+00  2.74e-01
  8  +1.9844e+02  +1.2821e+02  5.48e-01  3.70e-05  1.34e-05  1.94e+00  1.39e+00  5.26e-01
  9  +3.1007e+02  +2.5762e+02  2.04e-01  2.77e-05  1.00e-05  1.47e+00  1.08e+00  4.50e-01
 10  +5.4877e+02  +5.2321e+02  4.89e-02  1.35e-05  4.91e-06  7.98e-01  4.82e-01  8.25e-01
 11  +6.6921e+02  +6.5368e+02  2.38e-02  8.29e-06  3.00e-06  6.20e-01  2.64e-01  8.21e-01
 12  +8.6256e+02  +8.5765e+02  5.72e-03  2.61e-06  9.50e-07  1.87e-01  8.33e-02  9.90e-01
 13  +9.5348e+02  +9.5305e+02  4.48e-04  2.25e-07  8.28e-08  1.16e-02  7.38e-03  9.16e-01
 14  +9.6155e+02  +9.6153e+02  2.40e-05  1.21e-08  4.55e-09  5.14e-04  4.34e-04  9.43e-01
 15  +9.6194e+02  +9.6193e+02  4.17e-06  2.12e-09  7.97e-10  1.24e-04  7.72e-05  8.94e-01
 16  +9.6200e+02  +9.6200e+02  9.28e-07  4.73e-10  1.78e-10  2.94e-05  1.72e-05  8.16e-01
 17  +9.6202e+02  +9.6202e+02  1.64e-07  8.37e-11  3.15e-11  5.66e-06  3.05e-06  8.75e-01
 18  +9.6202e+02  +9.6202e+02  7.10e-08  3.01e-11  2.06e-11  2.67e-06  1.33e-06  7.57e-01
 19  +9.6202e+02  +9.6202e+02  5.29e-09  2.24e-12  1.51e-12  2.00e-07  9.89e-08  9.27e-01
-------------------------------------------------------------------------------------------
Terminated with status = Solved
solve time = 35.703515ms
  [CVXPYgen] obj=952.4242  time=0.0531s  eta=1.0 (compiled)
  [DIFF]     |cvxpy - cpg| = 0.036382  ✓ ~0
  [VERIFY]   Einit(bus2)=0.0950pu | cvxpy=952.46 | cpg=952.42 | prev_cpg=954.68 | cpg_changed=✓ YES
  Sent.
```

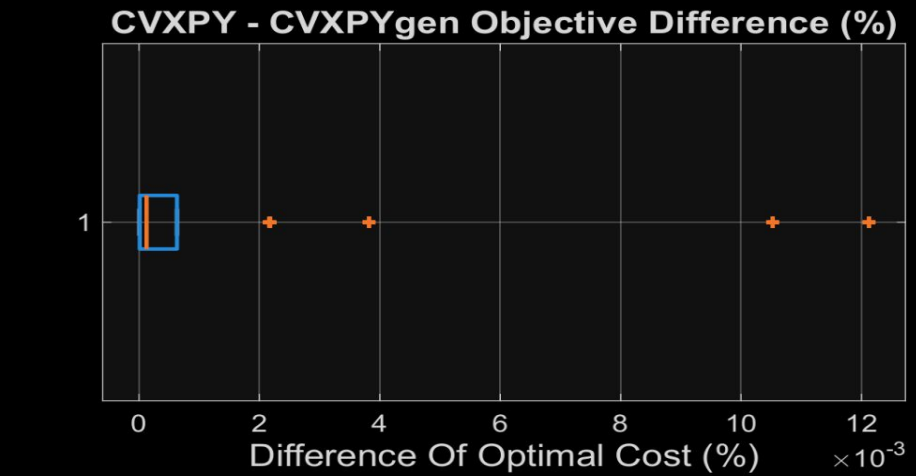
Command Window

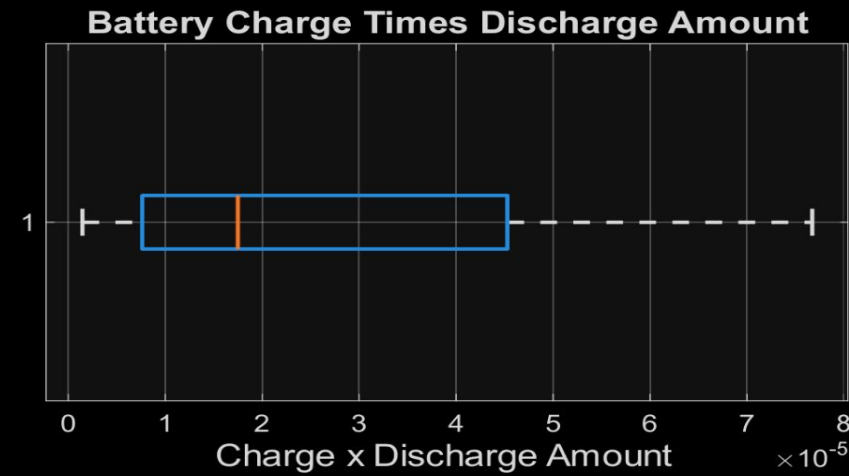
```
Permutation 296/304 (c=0.95, d=0.95, e=0.55)
Permutation 297/304 (c=0.95, d=0.95, e=0.60)
Permutation 298/304 (c=0.95, d=0.95, e=0.65)
Permutation 299/304 (c=0.95, d=0.95, e=0.70)
Permutation 300/304 (c=0.95, d=0.95, e=0.75)
Permutation 301/304 (c=0.95, d=0.95, e=0.80)
Permutation 302/304 (c=0.95, d=0.95, e=0.85)
Permutation 303/304 (c=0.95, d=0.95, e=0.90)
Permutation 304/304 (c=0.95, d=0.95, e=0.95)
Completed: 304/304 successful permutations.
```

Statistical/Performance Analysis - 304 Fixed Permutations









Key Findings, Critical Analysis And Theoretical Validation

01 Solver Fidelity & Speedup

- **Objective Difference:** Negligible diff **(<0.012%)** confirms CVXPYGEN matches standard CVXPY precision.
- **Approximately 25.7x Acceleration:** Speedup from 1.1s to ~50ms on average on Raspberry Pi via C-compilation, eliminating Python overhead from CVXPY.

02 Parameter Verification

- **Update Accuracy:** 304-run sweep matches prior fixed-parameter benchmarks (obj ≈ 990.72 at 0.01 p.u.).
- **Interface:** cpg_solve() correctly handles Einit, solar, and load profile updates per iteration.

03 Theorem 1 Validation

- **Complementarity:** Product p_c × p_d negligibly small across all 304 cases.
- **Implication:** Penalty costs implicitly enforce battery constraints, eliminating the need for mixed integer solvers.

04 Einit Economic Impact

- **Inverse Correlation:** Higher Einit leads to lower objective cost (952 vs 993) due to stored energy dispatch.
- **Grid Savings:** Full batteries reduce high-cost AC grid purchases at Bus 1.

Summary Metric: Cost Formulation (Based On My Interpretation)

Cost = Σ(net power injections) − total load + Σ(grid power × electricity price) + battery usage penalty

Successfully verified physically meaningful trends across 100% of the parameter space sweep.

Challenges Faced And Future Recommendations

ALGORITHMIC

Objective Gap Debugging

CVXPYGEN returned stale objectives due to cp.Parameter() not passing through to compiled C solver, causing a ~30 gap and 20% Clarabel failures. Setting **wrapper=True** in **generate_c_code.py** fixed the issue - achieving 100% pass rate with objectives correctly tracking Einit across all 304 permutations.

NETWORKING

Pi Connectivity & Build

WiFi manual config failed; resolved via reflash. Ethernet discovery issues bypassed using **IPv6 link-local**. Fixed CMake linker order (Clarabel → LAPACK → BLAS) using CMakeLists.txt when CVXPYGEN binary target **cpg_example** kept failing.

LEARNING

Theory & Code Ramping

Learnt about **Convex Optimization**, SDP formulations, CVXPY/CVXPYGEN algorithms implementation and multi-language stack developmentn(MATLAB/Python/C) within a tight 9-week timeline.

NEXT STEPS

Recommendations

1. **Simulink Integration** via C/S-Functions.
2. **Full η Sweep** after reformulating DPP constraints to study true efficiency impact.

My Overall Learnings And Strategic Insights

OPERATION LOGIC

The 24-hour microgrid cost is minimized by balancing **AC grid purchases** against solar generation and battery dispatch across Buses 1-5.

ECONOMIC DYNAMICS

Higher Einit directly correlates with lower costs. SOC depletion over time validates the battery effectively substituting for expensive grid power.

THEORETICAL INSIGHT

Complementarity is "free" via penalty costs (1×10^{-4}). Theorem 1 eliminates the need for complex mixed-integer solvers in this formulation.

HARDWARE STACK

CVXPYgen is the critical bridge for **SDP on embedded systems**. Unlike CVXGEN, it handles matrix variables and PSD constraints, enabling real-time deployment on Raspberry Pi.

AI FOR RESEARCH

Google search for background research before diving into Ariana's codebase, GenAI tools (Claude/Gemini) significantly **accelerated code debugging, generation and conceptual understanding of key concepts based on my initial understanding and readings after looking through the problem**, and cross-checked against actual numerical results which match.

Conclusion

DEPLOYMENT

Real-Time Ethernet Implementation

Translated multi-period SDP microgrid problem from **MATLAB/CVX to C**. Deployed on **Raspberry Pi 5** via real-time Ethernet interface.

PERFORMANCE

Approximately 25× Speedup Achieved

CVXPYGEN eliminated Python overhead, reducing latency with **<0.012% objective difference** across 304 test scenarios.

CORE VALIDATIONS

1. **Parameter Fidelity:** cpg_solve() updates correctly per iteration across 304 permutations
2. **Economic Logic:** Costs decrease monotonically with higher initial battery charge.
3. **Theorem 1 Success:** Complementarity **$p_c \times p_d = 0$** is satisfied implicitly without mixed-integer solvers.
4. **Future:** Target Simulink HIL integration and efficiency (η) sweeps.

Thank You!

QUESTIONS & DISCUSSION

Presented by:

Anurag Chatterjee

MPLab Research Project

May 14, 2026